

Proof of Secure Element Attestation: Sybil Resistance from Hardware Scarcity Rather Than Cost

Jan Kučera

NADO

admin@bismuth.cz

Abstract—Permissionless systems that reward participants rather than capital face an adversary ordinary countermeasures cannot price: the identity farm. Any admission rule charging a fixed cost per identity leaves the adversary’s cost-to-reward ratio equal to the honest participant’s, while the adversary alone amortises setup across its identities. On the deployment we report, some 1,000 of 1,191 identities belonged to two or three operators running browser farms and captured 42% of emission, after four such rules had been tried.

Proof of Secure Element Attestation (PoSEA) makes identities *scarce* rather than expensive, admitting a participant on a hardware attestation verified deterministically and offline against pinned vendor roots, one identity per physical device. We then address its largest gap: of 664 measured attempts, 26.8% came from machines with a healthy TPM 2.0 that no platform will certify, the vendor’s attestation-key service having no authority registered for the chip. Their vendor-signed *endorsement* key cannot sign, which is why the standard remedy is a privacy certificate authority holding a key able to assert any chip’s identity.

We give an enrolment protocol admitting such machines with no certificate authority and no long-lived key at all, resting on the observation that TPM2_MakeCredential’s credential blob is deterministic in its (seed, name, secret) triple: a challenge issued once is replayable offline by every verifier forever, so a commit–reveal across k beacon-drawn challengers, ordered by the ledger, supplies the freshness a signature would otherwise provide. Its security reduces to publication order and to one TPM object attribute. We validate it against a TPM 2.0 simulator and physical silicon, and state the cost: automatic enrolment removes the human tap, so identity price collapses onto the price of a chip.

Index Terms—Sybil resistance, hardware attestation, trusted platform module, permissionless consensus, commit–reveal

I. INTRODUCTION

A permissionless network that wishes to reward people rather than capital must decide who counts as a person. Proof of Work [1] answers by making participation expensive in energy; Proof of Stake answers by making it expensive in capital. Both are auctions, and in an auction the participant holding the most of the scarce input wins. Concentration follows by construction rather than by accident.

Systems that attempt to escape this by paying per participant inherit the Sybil problem [2] in its sharpest form: if an identity can be minted by a script, a reward paid per identity is a subsidy to whoever mints the most. Deployed responses have converged on two shapes. One imposes a recurring cost or delay on each

identity, which is the family we argue below cannot work. The other binds identities to something the adversary cannot replicate—a synchronous human ceremony in Idena’s case [7], a physical gathering in proof-of-personhood [4]—which does work, at the price of a ceremony every participant must attend.

We report a deployment in which the first family was tried four times and failed four times, give the measurements, and argue that the recurrence is structural rather than a sequence of implementation errors. PoSEA belongs to the second family, and substitutes a physically distinct *object* for the ceremony.

A. Contributions

- We state and measure the linearity argument that makes per-identity admission rules ineffective against identity farms (§II-B), with deployment data from a live chain on which four successive such rules failed.
- We specify PoSEA, an admission predicate based on secure-element attestation with deterministic, offline, replicable verification, and the per-device binding that makes “one device, one identity” enforceable (§III).
- We quantify the mechanism’s coverage gap: 26.8% of admission attempts are machines with functioning TPM 2.0 hardware that no platform will certify (§IV).
- We give a certificate-authority-free enrolment protocol closing that gap (§V). Its novelty is not cryptographic: it is the observation that a TPM credential blob is deterministic in its inputs, which converts an inherently interactive possession proof into an artifact any verifier can re-derive, so that ledger *ordering* can replace an issuer’s signature.
- We analyse its security (§VI), reporting that it reduces to publication order and to the restricted object attribute, and validate it against a TPM 2.0 simulator and physical silicon (§VIII).
- We state the costs without mitigation (§IX), including one the CA-free path introduces: it removes the human tap, so the security argument shifts from non-automatable human work to hardware price alone.

II. BACKGROUND AND THREAT MODEL

A. Deployment context

NADO is a permissionless chain with a two-lane block production schedule. A *bonded* lane weights producers by stake. An *open* lane, receiving 30% of slots, was designed so

a participant with no capital could produce blocks from an ordinary mobile device. Selection in both lanes is a deterministic beacon-keyed weighted draw: one hash determines each slot’s producer, so there is no grinding advantage and no hash race. Finality is a bonded-stake quorum with Casper-style epoch checkpoints [6].

The open lane was defended, successively, by a non-parallelisable sequential work proof at registration, a per-subnet ingress budget, a per-address identity cap, and a probationary period before a new identity could earn. Each was deployed in response to observed farming. Each failed.

B. Why per-identity rules fail

Observation 1 (Linearity): Let c be the cost an admission rule imposes per identity and r the reward per identity. An adversary controlling n identities pays nc and earns nr ; an honest participant pays c and earns r . The cost-to-reward ratio is c/r for both. Raising c therefore does not disadvantage the adversary relative to the honest participant; it disadvantages both equally, and prices out the honest participant first, because the adversary amortises fixed setup across n while the honest participant cannot.

Observation 1 is elementary. We state it explicitly because practice continues to propose per-identity rules as anti-Sybil measures, and because it identifies what a working defence must do: bound n itself, rather than tax it.

C. Measurement

On 2026-09-06 the open lane carried 1,191 registered identities. Approximately 1,000 were attributable to two or three operators running headless browser farms on rented Linux and Windows servers. The farmed set captured approximately 42% of total emission. Every honest participant was, in effect, subsidising three people.

D. Threat model

The adversary is computationally bounded, may hold arbitrary capital, may rent unlimited conventional compute, and controls every host it operates, including their operating systems. It may create unlimited network identities and key pairs at negligible cost, may observe all ledger state, and may participate in the protocol in any role including as a validator.

We assume the adversary cannot (i) forge a signature from a silicon vendor’s attestation root, (ii) extract a private key from a secure element it does not physically possess, or (iii) cause a genuine secure element to attest to a key that does not reside in it. Assumption (ii) is bounded, not absolute, and §X treats it as the construction’s principal open question; published attacks require physical access [18], [19], which prices extraction per device rather than granting a remote oracle—so far.

We do not assume any honest majority among challengers in §V; we assume only that not all k drawn challengers collude, and §VI makes the dependence explicit. We do not assume verifier clocks agree: all validity is evaluated at a ledger-agreed anchor time.

TABLE I
DEVICE CLASSES AND PER-DEVICE BINDING

Class	fmt	Bindable by
Android 12+, locked	android-key	RKP cert
Windows, physical TPM	tpm	AIK cert
Any TPM 2.0, no CA (§V)	—	endorsement key
FIDO2 security key	packed	<i>not bindable</i>
Apple Secure Enclave	apple	<i>no attestation</i>

III. POSEA

A. Intuition

Only two properties resist replication by an adversary with money but no people: capital, which the bonded lane already weighs, and a physically distinct object. PoSEA anchors open-lane identity in the second. A secure element is a physically distinct object that can attest to its own genuineness; it cannot be instantiated in software, and—the property that matters here—it cannot be instantiated in software by an adversary who already controls the host.

Definition 1 (PoSEA admission): An identity is admitted for one lease period if it presents a WebAuthn [8] attestation statement in which

- 1) a non-exportable key was generated inside a secure element;
- 2) that key signs a challenge derived from a ledger-selected beacon value, so the signature cannot be precomputed;
- 3) the attestation certificate chain terminates at a vendor root pinned as a protocol constant, every signature verifying at the anchor time;
- 4) the attested boot state indicates a locked bootloader and verified boot [9], where the class reports it; and
- 5) where the class exposes a per-device certificate, that certificate is not currently bound to another identity.

B. Device classes

Table I lists the classes considered. Four are reachable from a web page and differ in whether they yield a per-device handle, which decides whether condition (5) is enforceable at all; the fifth is the subject of §V and is reachable only from a native client.

With remote key provisioning (Android 12+), the certificate at `x5c[1]` is unique to the device and reused for every credential it creates until rotation—approximately two weeks in our sample. A pre-RKP device instead carries a multi-year intermediate shared by up to 10^5 units; the two are distinguished by validity span, and the batch case is refused, since binding a shared certificate to one identity would lock out every other owner of that model. On Windows, the attestation identity key (AIK) certificate is unique to the pair (TPM, account).

FIDO2 authenticators issue one batch certificate per $\geq 10^5$ units by design, as a privacy measure: they identify a model, not a unit, and are refused. Apple passkeys return `fmt: none` and carry no attestation through a web page (iOS 16+, macOS 13+, confirmed 2026-09-07); a native App Attest bridge is the

only route and is not deployed. What cannot be bound is not evidence of anything, so both are excluded rather than admitted with a weaker guarantee.

The third row of Table I is not a class any platform exposes. It is the path constructed in §V, reachable only from a native client, and it binds on the endorsement key—one per chip by manufacture, rather than one per account.

C. Determinism as a requirement, not a preference

In a replicated setting, admission is consensus input: every verifier must reach the same verdict on the same bytes, forever, including on replay years later. This forces four decisions that a single-verifier deployment could take differently.

Roots are pinned by SHA-256 over DER and stored with the protocol. They are never fetched, never learned from a peer, and change only by a gated commit. Revocation lists are deliberately not consulted, because doing so makes validation network-dependent and time-dependent; a compromised intermediate is handled as a root-set change. This trades revocation latency for deterministic replay, consciously.

Certificate validity is evaluated at a ledger-agreed anchor time rather than a wall clock. With wall clocks, a certificate expiring mid-block is valid on one verifier and expired on the next: a fork with no adversary involved.

The attestation parser must reject duplicate CBOR map keys. Permitting them allows a statement to verify as one chain and bind as another. This, and the anchor-time rule, correspond to holes found and closed in the deployment.

Finally, a binding must occupy its per-device uniqueness key *within* the block that creates it, so two provers cannot bind the same device in one block by each checking only against parent state.

D. What the mechanism does and does not prove

It proves the registration was produced on genuine hardware of a stated class, by a key resident in that hardware, over a challenge that could not be precomputed; and on renewal, that the same device key was present later.

It does not prove personhood. One human with one genuine device may create as many identities as they physically tap, because WebAuthn requires user activation per signature. The claim is that an identity costs *non-automatable human work on genuine hardware*, not that identities map one-to-one onto people; proof-of-personhood in the stronger sense remains a separate problem [4], [5]. Rooting is not a loophole, and is worth stating because it is the first objection raised: a rooted device is precisely what would automate the tap, and a rooted device cannot attest, because key attestation reports boot state from the secure element itself.

Block production is unchanged. Attestation decides membership of the draw, not its outcome, so faster hardware confers no advantage; attestation and presence weight are excluded from fork-choice weight and from the finality quorum. Presence earns reward and never purchases a say in what is final.

TABLE II
ADMISSION ATTEMPTS BY OUTCOME ($n = 664$)

Outcome	Share
Succeeds (phone / hardware wallet)	47.9%
TPM healthy, platform will not attest	26.8%
User-fixable (password manager took the ceremony)	18.8%
Succeeds (Windows Hello attests)	6.5%

IV. THE UNCERTIFIED-CHIP GAP

Table II gives the outcome of 664 consecutive admission attempts.

The second row is not a hardware fault. A WebAuthn tpm statement requires `certInfo` to be signed by the key in `x5c[0]`, so the chip needs an attestation-key *certificate*, which on Windows means the platform vendor’s attestation service. For a large population of otherwise healthy machines, that service has no certificate authority registered for the chip’s KeyId and answers HTTP 404. The vendor has confirmed the condition as service-side, with KeyIds not registrable by end users and no supported route for a user to alter the trust database. No client-side change repairs it. On Linux the situation is simpler and worse: there is no equivalent service and there has never been a path.

We established that the platform path could not be repaired from the client side by decoding the platform’s own key-attestation claim structure, which we document separately [21]: the platform API mints a fresh per-call signer whose certificate no relying party has ever seen, so no arrangement of client-side calls can satisfy WebAuthn’s requirement.

These machines are not short of evidence. They carry a vendor-signed *endorsement* certificate—AMD’s chaining to `CN=AMDTPM`, Intel’s through the CSME issuing authorities the chip holds in its own non-volatile memory. The hardware proof exists and is verifiable offline against pinned roots. What is missing is one statement: *this attestation key lives in that certified chip*.

V. ENROLMENT WITHOUT A CERTIFICATE AUTHORITY

A. Why the obvious remedies fail

Two proposals recur and both must be rejected.

Present the endorsement certificate as the proof. It fails twice: the certificate is public data, so anyone who has seen a machine’s certificate can present it; and the endorsement key *cannot sign*. TCG defines it as a restricted decryption key [11], and real certificates carry `Key Usage: critical`, `Key Encipherment` with `digitalSignature` absent. It can never occupy the `x5c[0]` slot.

Derive the challenge from public ledger data, so no round trip is needed. Anything every verifier can recompute, the prover can recompute. The resulting proof proves nothing.

Proving possession of a decryption-only key has exactly one shape: send it something only it can decrypt and observe it return. That is `TPM2_ActivateCredential` [10], and it is inherently interactive—which is why TCG specified the

attestation-key-plus-privacy-CA model, and why the failing service exists at all.

B. Replacing the authority with an ordering

A certificate authority converts the interactive proof into a durable artifact by signing it. That works, and it is expensive in a sense unrelated to computation: a long-lived key able to assert any endorsement identity is a key able to mint identities, and must be guarded as such for the lifetime of the system. In a permissionless setting it is also a single party the whole mechanism defers to, which is the objection the mechanism exists to answer.

Observation 2 (Replayable challenges): The credential blob returned by `TPM2_MakeCredential` is a deterministic function of the triple (seed, object name, secret). Only the OAEP-wrapped seed is randomised, and no verifier requires that half.

Observation 2 follows directly from the credential-protection construction of TPM 2.0 Part 1 §24 [10]: the blob is a symmetric encryption of the secret under a key derived from (seed, name) by KDFa, with an HMAC over the ciphertext and the name under a second derivation from the same pair. Every input is fixed by the triple. We confirmed it empirically: two calls with the same seed produce identical blobs and different wrapped seeds.

It is what removes the authority. A challenge issued once can be recomputed by every verifier afterwards, offline and indefinitely, from values that are public after the fact. The interactive proof therefore need not be signed into an artifact; it need only be *ordered*—and a ledger already orders things.

C. The protocol

Notation. E is a chip’s endorsement key and N the object name of an attestation key A in the same chip, N being a digest of A ’s public area. k is the number of challengers, S_i the secret challenger i seals and R_i the seed it seals under, b_i the resulting credential blob, and H a collision-resistant hash. *Ledger order* is the total order on published messages that every verifier agrees on; “strictly later” excludes equal positions.

Definition 2 (CA-free enrolment): Let E be a chip’s endorsement key, certified by a silicon vendor, and A an attestation key in the same chip with object name N . An enrolment is the following sequence of published messages, each strictly later in the ledger order than the message it answers:

- 1) the prover publishes E ’s certificate chain and A ’s public area, from which N is derived;
- 2) each of k drawn challengers publishes $b_i = \text{MakeCredential}(E, N, S_i; R_i)$ together with the wrapped seed;
- 3) the prover publishes $H(S_1 \parallel \dots \parallel S_k)$, having recovered every S_i by `TPM2_ActivateCredential`;
- 4) each challenger publishes (S_i, R_i) .

Verifiers accept when every b_i is reproduced by (S_i, R_i) under (E, N) and the commitment matches the concatenation in canonical challenger order.

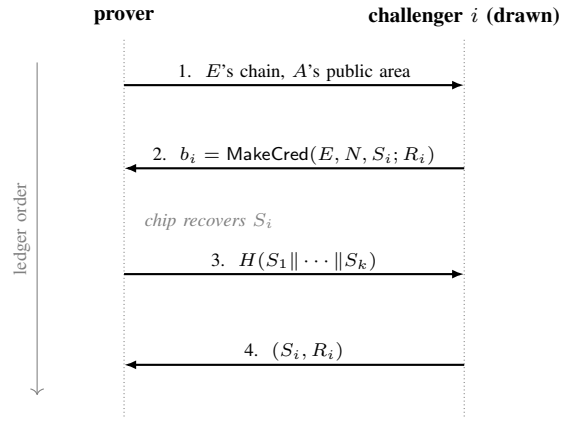


Fig. 1. The four messages. Each is published strictly later in the ledger order than the message it answers; steps 2 and 4 occur once per drawn challenger. Nothing is signed, and the ordering, not any signature, is what makes the exchange a proof (§VI).

Figure 1 shows the exchange. Nothing in Definition 2 is signed and no key persists between enrolments: there is nothing to steal, to rotate, or to guard, and no party holds an authority another party must trust.

D. Challenger selection

The residual attack is a challenger privately leaking S_i to a prover holding no chip. The defence is not to trust challengers, but to require several and to draw rather than accept them: the prover must recover every S_i , so forgery requires all k to collude. This replaces a key someone promises to protect with a property the system can observe.

The draw is weighted by a quantity the adversary cannot manufacture cheaply—in the deployment, bonded stake—and keyed on the same grind-resistant beacon that keys producer selection, so it is unpredictable before the enrolment is published and identical for every verifier. An unweighted draw over a permissionless set is the obvious error: cheap identities crowd the ballot and an adversary able to mint them draws its own challengers, restoring the attack in full. The draw is without replacement, since seating one challenger twice halves the collusion forgery requires. A short set is refused rather than accepted: an adversary able to shrink the challenger pool must not thereby reduce what forgery costs.

E. Binding and freshness

Identity binds to the *endorsement* key, never to an attestation key. A chip holds unlimited attestation keys, so binding to one would hand a single machine an identity per enrolment; the handle is SHA-256 over the endorsement certificate’s `SubjectPublicKeyInfo`. Regenerating the endorsement seed to fake a new chip changes that key, and the vendor’s certificate no longer matches it, so the fake cannot enrol at all.

An enrolment records that a key lives in a certified chip; it says nothing about when. Each admission therefore presents a fresh `TPM2_Certify` under the enrolled key whose `extraData` carries that admission’s own challenge. The

verifier additionally checks the `TPM_GENERATED` magic, the `TPM_ST_ATTEST_CERTIFY` structure type—a different type places different fields at the same offsets, so accepting one would mean parsing some other structure as this one—and that the certified object is the enrolled key rather than another object in the chip.

VI. SECURITY ANALYSIS

A. The ordering is the proof

Claim 1: The equations of Definition 2, verified without regard to publication order, are satisfiable by a prover holding no TPM.

Such a prover chooses S_i and R_i itself and computes b_i from the endorsement public key, which is public. Every check passes: each b_i is reproduced by its (S_i, R_i) , and the commitment matches by construction. Presented with all four messages simultaneously, a verifier cannot distinguish this from a genuine enrolment.

What separates them is that message 3 was published after every message 2 and before every message 4. The prover cannot learn S_i except by holding the chip, and must commit before the reveal, so it cannot read the answer off the ledger. Message 2 cannot precede message 1, because `MakeCredential` seals to a specific object name, which is a digest of the public area published in message 1. The four messages are therefore minimal: no adjacent pair may be merged.

Two consequences are easy to implement incorrectly. First, equal ordering positions are not ordered: two messages in one block have no order verifiers agree on, so every comparison must be strict. Second, the commitment must be over all k secrets jointly. One commitment per secret would let a prover answer whichever challenges it managed to open and abandon the rest, which is the k -of- k requirement dissolving into 1-of- k .

B. The attribute the argument rests on

Claim 2: If an unrestricted signing key may be enrolled, the certify of §V carries no information about where a key resides.

A `restricted` signing key signs only structures the TPM itself generated. That is what makes a certify evidence rather than merely a signature. An unrestricted key signs whatever its host composes, including a forged `TPMS_ATTEST` certifying a key that never existed in any chip; a verifier would then accept the forgery under a genuinely enrolled name. The public area is therefore validated before any challenge is issued, and must further carry `sensitiveDataOrigin`—absent it, the private half may have been generated outside and imported—and `fixedTPM` with `fixedParent`, absent which the key is duplicable to another chip and one enrolment would licence every machine it is copied to.

C. Attacks considered

Table III lists the attacks the construction refuses and the mechanism that refuses each. We include the case it does *not* refuse.

Collusion of all k drawn challengers admits a prover with no chip. We do not claim otherwise. The design intent is that this

TABLE III
ATTACKS AND THE MECHANISM THAT REFUSES THEM

Attack	Refused by
Replay a copied endorsement certificate	Possession proof: the seed unwraps only under the endorsement private key
Prover seals its own challenge	Challenger set is drawn, not chosen (§V-D)
Prover commits after seeing a reveal	Strict ordering, message 3 before 4
Prover opens some challenges, guesses the rest	Joint commitment over all k secrets
Challenger reveals a different secret	Blob already published; <code>MakeCredential</code> is deterministic
Enrol a second key for a second identity	Binding is on the endorsement key
Reuse an old certify	<code>extraData</code> carries the admission's own challenge
Certify some other object in the chip	Certified name must equal the enrolled name
Enrol a key that can sign arbitrary data	<code>restricted</code> required (Claim 2)
Duplicate an enrolled key to another chip	<code>fixedTPM</code> , <code>fixedParent</code> required
All k challengers collude	Not refused (§VI)
Physical key extraction	Not refused (§X)

is a condition the system can *observe*—the drawn set is public, weighted and beacon-keyed—rather than a key whose custody someone asserts. With $k = 3$, the smallest value at which no single challenger can admit identities alone, an adversary must both control challengers and win the draw for the specific enrolment it wishes to forge.

VII. IMPLEMENTATION

The reference implementation is dependency-free Python: `KDFA`, `AES-CFB`, `MGF1`, `RSA-OAEP`, `PKCS#1 v1.5` verification and the DER encoding are written out rather than imported. This is not minimalism for its own sake. A verifier that must agree byte-for-byte with every other verifier is poorly served by whichever version of a library each host happens to have installed, and in the deployment this code runs inside consensus on nodes whose runtime carries no cryptographic library at all. We record two obstacles that cost us time and that we expect to cost others the same.

A. Real endorsement certificates are not strictly DER

AMD encodes `critical: FALSE` explicitly where DER requires the default omitted. Python's `cryptography` refuses such a certificate outright with an `EncodedDefault` parse error, while `openssl` and Rust's `x509-parser` accept it. An implementer reaching for the obvious library will conclude the hardware is defective; it is not, and the certificate is the one the vendor issued. We moved endorsement-chain parsing to a lenient parser, shared by all verifiers.

B. A published “root” that is an intermediate

The certificate one vendor publishes as its endorsement root is cross-signed by a commercial certificate authority and is therefore an intermediate. Pinning it would silently relocate the trust decision to that authority, which is a different assertion from “this vendor manufactured this chip”. We verify self-signedness before pinning, and omit that vendor rather than pin an intermediate.

C. Signature verification

PKCS#1 v1.5 verification compares the entire re-encoded block [15]. An implementation that instead searches for the digest within the padding accepts trivial forgeries against small exponents—Bleichenbacher’s 2006 result [16], [17], which production libraries have shipped more than once. We pin this as a test rather than a comment.

VIII. EVALUATION

We exercised the construction against a TPM 2.0 simulator, driving our own command encodings over the simulator’s transport. The TCG endorsement template derives its 314-byte public area; a restricted RSA-2048 signing key derives; our challenger’s credential blob is accepted and opened by `ActivateCredential`, returning the sealed secret; an independent verifier re-derives the identical blob from the revealed pair; and a certify verifies, while a certify produced by a *different* key in the same chip is refused under the enrolled key’s public area. The full four-message enrolment reaches its proven state on values the chip actually produced. The enrolment has additionally been completed against a physical AMD firmware TPM.

Simulation does not establish that a particular vendor’s silicon behaves this way: a simulator accepts templates a firmware TPM may reject, and carries no endorsement certificate at all, so the one thing this path ultimately rests on—the vendor signature—cannot be exercised there. This is why we report the physical result separately and regard broader hardware coverage as required work rather than a formality.

IX. DISCUSSION

A. Vendor authority in the trust path

Verification requires trusting that vendor attestation roots sign only genuine hardware. This is a centralisation cost in a system whose premise is that no such authority should be necessary, and we do not think it can be argued away.

We observe only that every Sybil mechanism trusts something: Proof of Work trusts that no coalition controls a hash majority, an assumption weakening empirically; Proof of Stake trusts that concentrated holders do not collude. PoSEA trusts pinned vendor roots, verified offline, with no service able to revoke a participant at validation time. We regard this as a different trust assumption rather than a strictly worse one, and note that reasonable readers disagree.

B. Exclusion of modified devices

A device whose owner has unlocked its bootloader cannot attest. This excludes, disproportionately, the users most committed to the principle that an owner should control their own hardware—the constituency most sympathetic to a permissionless chain. The deployment mitigates this only partially, by a bonded lane requiring no device, which is a capital gate and therefore not a substitute.

C. What the CA-free path costs

The human tap is gone. Enrolment and renewal are automatic, which is required for headless machines and is why a server can participate at all; it also means a farm’s chips cost nothing to keep running. Identity *count* is unchanged—one chip, one identity, however many attestation keys it enrolls—but the security argument shifts. Where the WebAuthn path prices an identity at non-automatable human work on genuine hardware, this path prices it at the hardware alone.

The choice of pinned endorsement roots therefore becomes the entire dial, and it reverses only in the widening direction: narrowing the set later strands everyone already enrolled under the wider one. A discrete TPM module costs roughly an order of magnitude less than the machine a CPU-vendor firmware TPM implies. We regard this as a policy choice a deployment must make deliberately and state, not a parameter to be set by pinning every root that can be found.

X. OPEN PROBLEM

The security of the construction reduces to an empirical quantity we cannot presently bound: the cost of extracting an attestation or endorsement private key from a secure element, and whether that cost is specific to a model or generalises across a chip family.

The question is not open in the sense of untouched. TPM-FAIL [18] recovered ECDSA private keys from an Intel firmware TPM and an STMicroelectronics TPM by timing analysis; `faulTPM` [19] extracted secrets from AMD firmware TPMs by voltage fault injection against the AMD Secure Processor; and Samsung’s TrustZone keymaster was shown to permit attestation-relevant key compromise by design error [20]. These results matter to us directly: one of them targets a vendor whose root we pin.

What they establish is that extraction is possible with *physical access to the target machine*, which prices it per device and does not hand a remote adversary an oracle. What they do not establish—and what we need—is whether any of these generalises to a key that can be produced without possessing each chip. If extraction remains per-device and expensive, device scarcity holds and the adversary must buy hardware. If one extraction generalises across a family, the adversary obtains an oracle for minting attestations and the construction degrades to precisely the identity farm it was built to prevent, having excluded honest modified devices in the meantime.

This bears more heavily on the CA-free path than on the WebAuthn one, because that path has no human tap to slow an oracle down. We are not aware of a satisfying public treatment

at the granularity the design requires, and we regard establishing it as the most valuable next step. We would rather learn that the assumption fails than deploy on it silently.

XI. RELATED WORK

Sybil resistance. Douceur [2] establishes that without a trusted identity authority, an adversary can present arbitrarily many identities absent a resource constraint—and PoSEA is best read as choosing which resource. Social-graph defences [3] bound identities by trust-graph structure, requiring a graph the system must already possess. Proof-of-personhood [4], [5] binds identities to physical gatherings; PoSEA binds them to physical objects, which scales without ceremony and proves correspondingly less.

Attestation without an issuer. Direct Anonymous Attestation [12] solves an adjacent problem—removing the privacy CA from TPM attestation—by group signatures with an issuer that sets up the group. It provides unlinkability, which we do not, and requires an issuer and its setup, which we do not. Our construction removes the issuing party entirely, at the cost of interaction and of a public ordering; it is applicable precisely because a ledger supplies that ordering for free, and would not be applicable without one.

Remote attestation generally. SGX remote attestation [13] similarly terminates in a vendor service, and shares the failure mode we report: when the vendor service is unavailable to a party, the hardware’s evidence becomes unusable regardless of its integrity.

Commit–reveal. The commitment scheme is textbook [14]; the contribution is not the scheme but Observation 2, that a TPM credential blob is deterministic in its inputs, which is what makes a commit–reveal sufficient here. Without that, a challenge could not be replayed and every verifier would need the challenger’s word.

XII. CONCLUSION

Per-identity admission rules cannot solve identity farming, because their cost is linear in identities and therefore neutral between an honest participant and an adversary. The alternative is to change what an identity is. Anchoring it in a physically distinct secure element makes identity scarce rather than expensive.

For the quarter of machines whose platform will not certify their hardware, we show that the certificate authority the standard construction requires can be removed entirely, because a TPM credential blob is deterministic in its inputs and a ledger can supply ordering where an issuer supplied a signature. The security of that construction rests on publication order and on one object attribute, both of which a verifier checks itself.

The price is admitting vendor attestation authorities into the trust path, excluding owners who have taken control of their hardware, and—on the CA-free path—resting on hardware price rather than human effort. Whether that price is worth paying depends on an empirical question about secure-element extraction costs that remains, to our knowledge, open.

AVAILABILITY

A standalone reference implementation, the specification, the pinned roots and the tests are public at <https://github.com/hclivess/posea>; the enrolment of $\$V$ is `posea/tpm.py` and `posea/enrolment.py`, specified normatively in `SPEC.md §9`. The deployment it is extracted from is at <https://github.com/hclivess/nado>. The reverse-engineered platform key-attestation claim format is documented at <https://github.com/hclivess/windows-tpm-claim-format>. The network is pre-mainnet; there is no token sale and nothing offered for purchase.

REFERENCES

- [1] S. Nakamoto, “Bitcoin: A peer-to-peer electronic cash system,” 2008.
- [2] J. R. Douceur, “The Sybil attack,” in *Proc. 1st Int. Workshop on Peer-to-Peer Systems (IPTPS)*, 2002, pp. 251–260.
- [3] H. Yu, M. Kaminsky, P. B. Gibbons, and A. Flaxman, “SybilGuard: Defending against Sybil attacks via social networks,” in *Proc. ACM SIGCOMM*, 2006.
- [4] M. Borge, E. Kokoris-Kogias, P. Jovanovic, L. Gasser, N. Gailly, and B. Ford, “Proof-of-personhood: Redemocratizing permissionless cryptocurrencies,” in *IEEE European Symp. on Security and Privacy Workshops (EuroS&PW)*, 2017.
- [5] B. Ford and J. Strauss, “An offline foundation for online accountable pseudonyms,” in *Proc. 1st Workshop on Social Network Systems (SocialNets)*, 2008.
- [6] V. Buterin and V. Griffith, “Casper the friendly finality gadget,” arXiv:1710.09437, 2017.
- [7] Idena, “Idena: Proof-of-person blockchain.” [Online]. Available: <https://idena.io>
- [8] W3C, “Web Authentication: An API for accessing public key credentials,” W3C Recommendation. [Online]. Available: <https://www.w3.org/TR/we-bauthn-3/>
- [9] Google, “Verifying hardware-backed key pairs with key attestation,” Android Developers. [Online]. Available: <https://developer.android.com/privacy-and-security/security-key-attestation>
- [10] Trusted Computing Group, “Trusted Platform Module Library, Part 1: Architecture” and “Part 3: Commands,” Family 2.0, Level 00, Rev. 01.59, 2019.
- [11] Trusted Computing Group, “TCG EK Credential Profile for TPM Family 2.0,” Level 0, Version 2.5.
- [12] E. Brickell, J. Camenisch, and L. Chen, “Direct anonymous attestation,” in *Proc. 11th ACM Conf. on Computer and Communications Security (CCS)*, 2004, pp. 132–145.
- [13] V. Costan and S. Devadas, “Intel SGX explained,” IACR Cryptology ePrint Archive, Report 2016/086, 2016.
- [14] M. Blum, “Coin flipping by telephone: A protocol for solving impossible problems,” in *Proc. CRYPTO*, 1981, pp. 11–15.
- [15] K. Moriarty, B. Kaliski, J. Jonsson, and A. Rusch, “PKCS #1: RSA cryptography specifications version 2.2,” RFC 8017, IETF, 2016.
- [16] H. Finney, “Bleichenbacher’s RSA signature forgery based on implementation error,” openssl-dev mailing list, Aug. 2006.
- [17] “CVE-2006-4339: OpenSSL RSA signature forgery,” 2006.
- [18] D. Moghimi, B. Sunar, T. Eisenbarth, and N. Heninger, “TPM-FAIL: TPM meets timing and lattice attacks,” in *Proc. 29th USENIX Security Symp.*, 2020.
- [19] H. N. Jacob, C. Werling, R. Bühren, and J.-P. Seifert, “faulTPM: Exposing AMD firmware TPMs,” arXiv:2304.14717, 2023.
- [20] A. Shakevsky, E. Ronen, and A. Wool, “Trust dies in darkness: Shedding light on Samsung’s TrustZone keymaster design,” in *Proc. 31st USENIX Security Symp.*, 2022.
- [21] J. Kučera, “Windows TPM key-attestation claim format,” 2026. [Online]. Available: <https://github.com/hclivess/windows-tpm-claim-format>